- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 2: Interacting with Web APIs

Problem Statement: Analyzing Weather Data from OpenWeatherMap API

Prerequisites

pip install requests pandas numpy matplotlib seaborn folium

```
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages
(2.32.4)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages
(0.13.2)
Requirement already satisfied: folium in /usr/local/lib/python3.13/dist-packages (0.20.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.13/dist-packages (from requests) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages
(from requests) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.13/distpackages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.13/distpackages (from requests) (2026.7.22)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.13/distpackages (from matplotlib) (1.5.1)
```

```
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)


Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.13/dist-packages
(from folium) (0.8.2)
Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.13/dist-packages
(from folium) (3.1.6)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.13/dist-packages
(from folium) (2026.9.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages
(from jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

Implementation Pipeline (Tasks 2 to 9)

```
import requests import pandas as pd
import numpy as np import
matplotlib.pyplot as plt import seaborn
as sns import folium from datetime
import datetime, timedelta
```

TASK 1 & 2: API Setup & Data Retrieval

```
API_KEY = "                                                    "
TARGET_CITY = "Pune,IN"
FORECAST_URL = "https://api.openweathermap.org/data/2.5/forecast"

def fetch_weather_forecast(city, api_key):
    """Fetches 5-day / 3-hour weather forecast data from OpenWeatherMap API."""
params = {
        'q': city,
        'appid': api_key,
        'units': 'metric'
    }
try:
        response = requests.get(FORECAST_URL, params=params, timeout=10)
if response.status_code == 200:            print(f"Successfully retrieved live API
data for {city}.")            return response.json()        else:
            print(f"API returned status code {response.status_code} ({response.json().get('message')}).")
print("Switching to simulated OpenWeatherMap JSON response for seamless execution...")
return generate_fallback_json(city)        except Exception as e:            print(f"Network error ({e}).
Switching to simulated JSON response...")        return generate_fallback_json(city)

def generate_fallback_json(city):
    """Generates an exact replica of OpenWeatherMap's 5-day/3-hour JSON schema."""
np.random.seed(42)
    base_time = datetime.now().replace(minute=0, second=0, microsecond=0)
forecast_list = []     for i in range(40):        dt_obj = base_time +
```

```
timedelta(hours=3 * i)       hour_factor = np.sin((dt_obj.hour - 6) * np.pi /
12)
    temp = round(26 + 5 * hour_factor + np.random.normal(0, 0.8), 2)
    humidity = int(np.clip(65 - 20 * hour_factor + np.random.normal(0, 5), 25, 98))
    rain_val = round(np.random.exponential(1.8), 2) if humidity > 75 and np.random.rand() > 0.4 else
0.0

    entry = {
      'dt_txt': dt_obj.strftime('%Y-%m-%d %H:%M:%S'),
      'main': {
        'temp': temp,
        'feels_like': round(temp + (0.05 * humidity - 2), 2),
        'temp_min': round(temp - np.random.uniform(0.5, 1.2), 2),
        'temp_max': round(temp + np.random.uniform(0.5, 1.2), 2),
        'pressure': int(np.random.normal(1011, 3)),
        'humidity': humidity
      },
      'wind': {'speed': round(np.random.uniform(1.2, 7.5), 2)},
      'clouds': {'all': int(np.clip(humidity * 0.9, 0, 100))},
      'weather': [{'main': 'Rain' if rain_val > 0 else 'Clouds', 'description': 'light rain' if rain_val > 0 else
'scattered clouds'}]
    }

    if rain_val > 0:       entry['rain']
=              {'3h':              rain_val}
forecast_list.append(entry)

  return {'city': {'name': 'Pune', 'coord': {'lat': 18.5204, 'lon': 73.8567}}, 'list': forecast_list}

raw_json = fetch_weather_forecast(TARGET_CITY, API_KEY) Successfully
retrieved live API data for Pune,IN.
```

TASK 3: Extract Relevant Weather Attributes

```
extracted_records = [] for item in raw_json['list']:
extracted_records.append({
    'Timestamp': item.get('dt_txt'),
    'Temperature_C': item['main'].get('temp'),
    'Feels_Like_C': item['main'].get('feels_like'),
    'Temp_Min_C': item['main'].get('temp_min'),
    'Temp_Max_C': item['main'].get('temp_max'),
    'Humidity_Pct': item['main'].get('humidity'),
    'Pressure_hPa': item['main'].get('pressure'),
    'Wind_Speed_ms': item['wind'].get('speed'),
    'Cloudiness_Pct': item['clouds'].get('all'),
    'Precipitation_mm': item.get('rain', {}).get('3h', np.nan),
    'Weather_Condition': item['weather'][0].get('main')
  })

weather_df = pd.DataFrame(extracted_records)
```

TASK 4: Clean and Preprocess the Data

```
# 1. Convert Timestamp string to standardized pandas datetime object
weather_df['Timestamp'] = pd.to_datetime(weather_df['Timestamp'])

# 2. Handle missing values: OpenWeatherMap leaves out 'rain.3h' when rainfall is 0
weather_df['Precipitation_mm'] = weather_df['Precipitation_mm'].fillna(0.0)

# 3. Interpolate any unexpected missing sensor readings in continuous columns
numeric_cols = ['Temperature_C', 'Feels_Like_C', 'Humidity_Pct', 'Wind_Speed_ms', 'Pressure_hPa']
weather_df[numeric_cols] = weather_df[numeric_cols].interpolate(method='linear')

# 4. Sort chronologically and drop duplicate timestamps if any weather_df
=
weather_df.sort_values('Timestamp').drop_duplicates(subset=['Timestamp']).reset_index(drop=True)

print("\n--- Task 3 & 4: Cleaned Weather DataFrame ---")
print(weather_df.info()) print(weather_df.head(3))
```

```
--- Task 3 & 4: Cleaned Weather DataFrame ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39 Data
columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
0   Timestamp          40 non-null     datetime64[ns]
1   Temperature_C      40 non-null     float64
2   Feels_Like_C       40 non-null     float64
3   Temp_Min_C         40 non-null     float64
4   Temp_Max_C         40 non-null     float64
5   Humidity_Pct       40 non-null     int64
6   Pressure_hPa       40 non-null     int64
7   Wind_Speed_ms      40 non-null     float64
8   Cloudiness_Pct     40 non-null     int64
9   Precipitation_mm   40 non-null     float64        10  Weather_Condition  40 non-null
    object         dtypes: datetime64[ns](1), float64(6), int64(3), object(1) memory
    usage: 3.6+ KB None
            Timestamp  Temperature_C  Feels_Like_C  Temp_Min_C  Temp_Max_C  \
0 2026-10-18 18:00:00          27.86         28.94       27.86       28.95
1 2026-10-18 21:00:00          28.15         29.44       28.15       28.72    2
  2026-10-19 00:00:00          26.04         26.04       25.13       26.04

   Humidity_Pct  Pressure_hPa  Wind_Speed_ms  Cloudiness_Pct  \
0            57          1014           1.96             100
1            58          1013           1.42             100    2
             65          1013           0.43              85

   Precipitation_mm Weather_Condition
0               0.0            Clouds
```

```
1                0.0            Clouds
2                0.0            Clouds
```

TASK 5: Data Modeling & Pattern Analysis

```
weather_df['Temp_Rolling_24h_Avg'] = weather_df['Temperature_C'].rolling(window=8,
min_periods=1).mean()
time_index = np.arange(len(weather_df))
slope, intercept = np.polyfit(time_index, weather_df['Temperature_C'], 1) weather_df['Linear_Trend']
= slope * time_index + intercept

print("\n--- Task 5: Overall Weather Pattern Summary ---") print(f"Average
Temperature : {weather_df['Temperature_C'].mean():.2f} °C") print(f"Maximum
Temperature : {weather_df['Temperature_C'].max():.2f} °C (at
{weather_df.loc[weather_df['Temperature_C'].idxmax(), 'Timestamp']})")
print(f"Minimum Temperature : {weather_df['Temperature_C'].min():.2f} °C (at
{weather_df.loc[weather_df['Temperature_C'].idxmin(), 'Timestamp']})")
print(f"Average Humidity    : {weather_df['Humidity_Pct'].mean():.1f} %")
print(f"Total Precipitation : {weather_df['Precipitation_mm'].sum():.2f} mm")
print(f"Max Wind Speed      : {weather_df['Wind_Speed_ms'].max():.2f} m/s")
```

```
--- Task 5: Overall Weather Pattern Summary ---
Average Temperature : 28.66 °C
Maximum Temperature : 34.91 °C (at 2026-10-19 09:00:00)
Minimum Temperature : 22.66 °C (at 2026-10-23 00:00:00)
Average Humidity    : 51.3 %
Total Precipitation : 0.12 mm
Max Wind Speed      : 3.58 m/s
```

TASK 7: Data Aggregation by Time Periods

```
weather_df['Date'] = weather_df['Timestamp'].dt.date weather_df['Hour']
= weather_df['Timestamp'].dt.hour

# Categorize hours into parts of the day def
assign_time_of_day(hour):    if 6 <= hour <
12: return '1_Morning'    elif 12 <= hour <
17: return '2_Afternoon'    elif 17 <= hour <
21: return '3_Evening'    else: return
'4_Night'

weather_df['Time_Of_Day'] = weather_df['Hour'].apply(assign_time_of_day)

# 1. Daily Aggregation daily_summary =
weather_df.groupby('Date').agg(
  Avg_Temp_C=('Temperature_C', 'mean'),
  Min_Temp_C=('Temp_Min_C', 'min'),
  Max_Temp_C=('Temp_Max_C', 'max'),
  Avg_Humidity_Pct=('Humidity_Pct', 'mean'),
```

```
    Total_Rain_mm=('Precipitation_mm', 'sum'),
    Max_Wind_ms=('Wind_Speed_ms', 'max')
).round(2).reset_index()

print("\n--- Task 7A: Daily Aggregated Weather Statistics ---")
print(daily_summary.to_string(index=False))

# 2. Time-of-Day Aggregation
tod_summary = weather_df.groupby('Time_Of_Day')[['Temperature_C', 'Humidity_Pct',
'Wind_Speed_ms']].mean().round(2)
print("\n--- Task 7B: Diurnal (Time-of-Day) Aggregation ---") print(tod_summary)
```

```
--- Task 7A: Daily Aggregated Weather Statistics ---
      Date  Avg_Temp_C  Min_Temp_C  Max_Temp_C  Avg_Humidity_Pct  Total_Rain_mm
Max_Wind_ms
2026-10-18       28.00       27.86       28.95             57.50           0.00
1.96
2026-10-19       30.74       25.13       34.91             52.75           0.00
3.54
2026-10-20       28.89       24.17       33.96             51.38           0.12
2.94
2026-10-21       29.12       26.08       33.89             49.88           0.00
3.58
2026-10-22       26.34       23.32       28.37             52.50           0.00
2.38
2026-10-23       28.28       22.66       32.51             47.33           0.00
2.40

--- Task 7B: Diurnal (Time-of-Day) Aggregation ---
             Temperature_C  Humidity_Pct  Wind_Speed_ms
Time_Of_Day
1_Morning            32.03          38.2           2.56
2_Afternoon          29.17          50.2           1.69
3_Evening            27.01          60.0           1.76 4_Night
26.62          57.8           1.25
```

TASK 6 & 9: Visualizations & Correlation Analysis

```
sns.set_theme(style="whitegrid") fig, axes = plt.subplots(2, 2,
figsize=(16, 11))
fig.suptitle(f'OpenWeatherMap Data Modeling & Analysis ({TARGET_CITY})', fontsize=16,
fontweight='bold')

# Plot 1 (Line Chart): Temperature Changes & Rolling Trend Over Time axes[0,
0].plot(weather_df['Timestamp'], weather_df['Temperature_C'], marker='o', label='3-Hourly Temp
(°C)', color='#e63946', alpha=0.8)
axes[0, 0].plot(weather_df['Timestamp'], weather_df['Temp_Rolling_24h_Avg'], linestyle='--',
linewidth=2.5, label='24h Rolling Mean', color='#1d3557') axes[0, 0].set_title('Temperature
Trend Over Time') axes[0, 0].set_xlabel('Date & Time') axes[0, 0].set_ylabel('Temperature
(°C)') axes[0, 0].tick_params(axis='x', rotation=30) axes[0, 0].legend()
```

```
# Plot 2 (Bar Plot): Daily Total Precipitation Levels
sns.barplot(    data=daily_summary,    x='Date',
y='Total_Rain_mm',    hue='Date',
palette='Blues_d',    ax=axes[0, 1],
legend=False
)
axes[0, 1].set_title('Daily Accumulated Precipitation (mm)')
axes[0, 1].set_xlabel('Date') axes[0, 1].set_ylabel('Total
Rainfall (mm)') axes[0, 1].tick_params(axis='x',
rotation=15)

# Plot 3 (Scatter Plot): Temperature vs Humidity colored by Wind Speed
scatter = axes[1, 0].scatter(    weather_df['Temperature_C'],
weather_df['Humidity_Pct'],    c=weather_df['Wind_Speed_ms'],
s=weather_df['Cloudiness_Pct'] + 30,
   cmap='viridis',
alpha=0.85,
edgecolors='k'
)
cbar = plt.colorbar(scatter, ax=axes[1, 0]) cbar.set_label('Wind
Speed (m/s)')
axes[1, 0].set_title('Temperature vs. Humidity (Color = Wind Speed, Size = Clouds)')
axes[1, 0].set_xlabel('Temperature (°C)') axes[1, 0].set_ylabel('Relative Humidity
(%)')

# Plot 4 (Correlation Heatmap - Task 9): Relationships Between Weather Attributes
corr_features = ['Temperature_C', 'Feels_Like_C', 'Humidity_Pct', 'Pressure_hPa', 'Wind_Speed_ms',
'Precipitation_mm']
corr_matrix = weather_df[corr_features].corr() sns.heatmap(
   corr_matrix,
annot=True,
fmt=".2f",
cmap='coolwarm',
   vmin=-1,
vmax=1,
ax=axes[1, 1]
) axes[1, 1].set_title('Correlation Matrix of Weather
Attributes')

plt.tight_layout() plt.show()
```

OpenWeatherMap Data Modeling & Analysis (Pune,IN)







